Introduction to Backend, APIs, RESTful APIs & API Testing

Table of Contents

Objective

Outline

Introduction

Module 1: Introduction to Backend Development

- 1.1 What is Backend Development?
- 1.2 Core Components of a Backend System
- 1.3 How the Web Works (Foundations)
- 1.4 Common Backend Responsibilities
- 1.5 The Backend Ecosystem (Overview)

Module 2: What is an API?

- 2.1 Definition & Intuition
- 2.2 How APIs Work
- 2.3 Types of APIs
- 2.4 Why APIs Matter
- 2.5 Key API Concepts

Module 3: RESTful APIs & Client-Server Architecture

- 3.1 Client-Server Architecture
- 3.2 What is REST?
- 3.3 REST Guiding Principles (Constraints)
- 3.4 HTTP Fundamentals for REST
- 3.5 Designing RESTful Endpoints
- 3.6 Data Exchange in REST
- 3.7 API Authentication & Security (Intro)

Module 4: API Testing

- 4.1 Why Test APIs?
- 4.2 Types of API Testing
- 4.3 What to Validate in an API
- 4.4 Anatomy of an API Test
- 4.5 Tools for API Testing (Overview)

4.6 Introduction to Test Automation
4.7 Best Practices
Hands-On Practice
Reference Material
Advanced Reading Material
Key Takeaways

Introduction to Backend, APIs, RESTful APIs & API Testing

Objective

By the time you finish this guide, you should have a clear picture of what a backend actually does and how the web carries data around behind the scenes. You will be able to explain what an API is and why we bother having them, read and design a simple REST API without getting lost, and test an API to check that it behaves the way you expect. The aim is to give you a solid mental model you can keep building on, not to make you memorise the syntax of any one language or tool.

Outline

Here is what we will walk through:

- **Module 1, Introduction to Backend Development.** what the backend is responsible for, the parts it is made of, how the web works underneath it, and the wider ecosystem.
- **Module 2, What is an API.** a plain-English definition, how APIs actually work, the different kinds you will run into, and why they matter so much.
- **Module 3, RESTful APIs and Client-Server Architecture.** the client-server model, what REST really means, its ground rules, the HTTP you need, and how to design and secure endpoints.
- **Module 4, API Testing.** why we test, the types of testing, what to check, how a test is put together, the common tools, and good habits.

Introduction

Think about any app you used this morning: a banking app, a shopping site, a food-delivery service. Every one of them really has two halves. There is the part you can see and tap, the screens and buttons, and there is the part you never see, the machines somewhere that hold your data, run the rules, and keep everything safe. That hidden half is the backend, and the thing that lets the two halves talk to each other is the API.

This guide builds up from the ground. We start with what a backend is and how a single request travels across the internet and comes back. Then we look at APIs as the doorways a backend opens up to the outside world. From there we spend time on REST, which is by far the most common style of web API, and we finish by learning how to test an API so you can actually trust it. Throughout, the focus stays on ideas that hold true no matter which language or framework your team happens to use.

Who this is for: new joiners, and anyone shifting into backend work. We assume no backend experience. If you have used websites and written a little code, you are ready.

Module 1: Introduction to Backend Development

1.1 What is Backend Development?

Backend development is the work of building the server side of an application: the logic, the data, and the services that run on a machine the user never sees directly. If the frontend is the shop counter that customers walk up to, the backend is the stockroom, the till, and the accountant working quietly behind the wall.

It helps to be clear on three related terms:

- **Frontend.** This is everything that runs in the user's browser or on their phone: the layout, the styling, and the interactions built with HTML, CSS, JavaScript, or a mobile toolkit. Its job is how the product looks and feels.
- **Backend.** This is everything that runs on the server: the business logic, the databases, the login checks, and the connections to other systems. Its job is to keep things correct, safe, and consistent.
- **Full-stack.** This describes a person or role comfortable working on both halves and, just as importantly, on the seam where they meet.

So what is the backend actually on the hook for? Broadly, three things. First, the business logic, which is the set of rules that make your product what it is, such as “a customer can only withdraw money if they have enough balance” or “this discount applies to first-time buyers only.” Second, the data: storing it, fetching it, and updating it so that it survives restarts and looks the same to everyone. Third, security: deciding who is allowed to do what, guarding secrets, and double-checking everything that arrives from the outside world.

Almost everything a backend does follows the same simple loop, which is worth committing to memory:

1. A client, such as a browser or a mobile app, sends a request over the network.
2. The server receives it, works out what is being asked, and runs the right piece of logic.
3. Along the way it may read from or write to a database.
4. It puts together a response, usually some data plus a status, and sends it back.

Mental model: the backend is a machine that turns requests into responses, applying rules and touching data in between. Hold on to this loop, because every later topic is really just a detail of one of these steps.

1.2 Core Components of a Backend System

A backend is rarely one single thing. It is usually a handful of parts that cooperate, and naming those parts makes the whole system far easier to reason about.

- **Server.** A program, running on a real or virtual machine, that listens for incoming requests and answers them. In casual conversation “server” sometimes means the hardware, but day to day it usually means the software process doing the serving.
- **Application, or business-logic, layer.** The code that decides what to do with a request: checking the input, applying the rules, and coordinating the steps. This is where most of your day as a backend developer is spent.
- **Database.** The part that stores data safely and lets you get it back quickly. It outlives any single request and acts as the system's long-term memory.
- **APIs.** The well-defined entry points through which the outside world, and other services, can use what the backend offers without needing to know how it works inside.

A request flows through these parts in order. It lands on the server, gets routed to the application layer, which may consult the database, and finally a response shaped by the API travels back out.

```
Client  ->  [ Server ]  ->  [ Application / Business Logic ]  ->  [ Database ]
                                        |
                                        v
Client  <-  [ Response via API ]  <-----+
```

1.3 How the Web Works (Foundations)

Backends live on the internet, so a working picture of how the web moves data around will save you a lot of confusion later.

- **Client, server, and network.** The client asks, the server answers, and the network is the set of roads between them. Your browser is a client, the machine hosting a website runs a server, and the internet is the network in the middle.
- **DNS, IP addresses, and domains.** Every machine on the internet has an IP address, a numeric identity such as 142.250.72.14. Numbers are hard to remember, so we use friendly domain names like example.com instead. DNS, the Domain Name System, is the phone book that looks up a domain and hands back its IP address so the request can find its way.
- **HTTP and HTTPS.** HTTP is the language clients and servers use to exchange requests and responses on the web. HTTPS is the same thing with encryption added, so nobody in between can read or tamper with the conversation.
- **Ports and localhost.** One machine can run many services at once, so each one listens on a numbered port: 80 for HTTP, 443 for HTTPS, or something like 8080 for a development server. The name localhost, address 127.0.0.1, is a shorthand your computer uses to mean “this machine,” which is how you reach a server you are running on your own laptop, for example http://localhost:8080.

> **The whole trip in one line:** you type a URL, DNS turns the domain into an IP address, your browser opens an HTTP or HTTPS connection to that address on a port, and the server responds. That is the entire journey.

1.4 Common Backend Responsibilities

Beyond simply returning some data, a backend is expected to handle a familiar set of concerns. You will meet these again and again, so it pays to recognise them early.

- **Authentication and authorization.** Authentication answers the question “who are you?”, which is the login step. Authorization answers “what are you allowed to do?”, which is about permissions. They are two separate checks, and both matter.
- **Data validation and processing.** Never trust anything that comes from outside. Confirm that it is present, well-formed, and within sensible limits before you act on it, then reshape it into whatever your system needs.
- **Business rules and workflows.** These are the multi-step processes that make the product tick, like placing an order, transferring money, or onboarding a new user. Often they pull together several pieces of data and a few outside services.
- **Logging, error handling, and performance.** Keep a record of what happened for debugging and audits, fail gracefully with a clear message instead of crashing, and stay fast enough to feel responsive even when a lot of people show up at once.

> **Rule of thumb:** if something must always happen, such as an input check, a permission check, or an audit log, make it a deliberate and tested part of the flow rather than an afterthought.

1.5 The Backend Ecosystem (Overview)

You do not need to learn all of this right now. The goal is just to recognise the landscape and see where each piece sits.

- **Languages.** Plenty of languages can build a backend, and each has its sweet spot: Java for large enterprise systems, Python for writing things quickly and for data or AI work, Node.js and JavaScript for sharing one language with the frontend, and Go for simple, fast services. C#, Ruby, and Rust are common too.
- **Databases.** These fall into two broad families. SQL, or relational, databases such as PostgreSQL and MySQL store structured rows in tables and give you strong consistency. NoSQL databases such as MongoDB, Cassandra, and Redis give up some of that structure in exchange for flexibility or scale. There is a quick comparison below.
- **Servers and runtimes.** This is the environment your code runs inside: a web server or application runtime like a JVM or a Node runtime, usually sitting behind a load balancer, and more and more often packaged in containers with Docker and managed with Kubernetes.
- **Where APIs fit in.** Whatever language or database you pick, the backend shows off its abilities through APIs. The API is the stable contract, and the technology behind it can change without breaking the clients that depend on it.

SQL and NoSQL at a glance:

Aspect	SQL (Relational)	NoSQL (Non-relational)
Data shape	Structured tables, fixed schema	Flexible documents, key-value, graph, and more
Consistency	Strong, transactional (ACID)	Often eventual, varies by store
Best for	Related data and complex queries	High scale or changing, unstructured data
Examples	PostgreSQL, MySQL	MongoDB, Redis, Cassandra

Module 2: What is an API?

2.1 Definition & Intuition

API stands for Application Programming Interface. Stripped of the jargon, it is simply an agreed way for one piece of software to ask another piece of software to do something, or to hand over some data, without needing to know how that other piece works on the inside.

Two everyday comparisons make this click. The first is a restaurant. You do not march into the kitchen and cook; you give your order to the waiter, who carries it to the kitchen and brings your food back out. The waiter is the API, the kitchen is the server, and the menu is the list of things you are allowed to ask for. The second is a power socket. Any appliance with the right plug just works, because the socket is a standard interface. You never think about the power grid; you simply rely on the promise the socket makes.

That word “promise” is the whole point. An API hides the messy details on one side and makes a clear promise about how to interact with it on the other. As long as that promise holds, either side is free to change internally without breaking the other. That is why we say APIs give us abstraction and a contract.

2.2 How APIs Work

Under the hood, an API runs on the same request-and-response loop from Module 1, just with a more formal shape. The client sends a request describing what it wants, the server processes it, and a response comes back.

An API exposes specific addresses called endpoints, and each one stands for a capability or a resource, for example /users or /orders/42. Calling an endpoint is how you trigger that capability. The flow looks like this:

1. The client calls an endpoint, optionally sending some data and proving who it is.
2. The server checks the request, runs the relevant logic, and touches data if it needs to.
3. The server sends back a response: the data that was asked for, plus a status that says whether things went well.

```
GET /weather?city=Pune        <-- request (asking for data)
```

```
200 OK
{ "city": "Pune", "tempC": 29, "condition": "Clear" }   <-- response
```

2.3 Types of APIs

APIs get sorted in two useful ways. The first is by who is allowed to use them:

- **Public, or open.** Available to outside developers, sometimes after signing up for a key. A public maps or payments API is a good example.
- **Private, or internal.** Used only inside one organisation, between its own systems.
- **Partner.** Shared with specific business partners under an agreement, and not open to the general public.

The second way is by the style or protocol they are built on. Here is a quick comparison of the four you are most likely to hear about:

Style	In one line	Typical use
REST	Resources over HTTP, usually JSON	Most web and mobile APIs (our focus)
SOAP	Older, XML-based, strict contracts	Legacy enterprise systems, banking
GraphQL	The client asks for exactly the fields it wants	Flexible data-fetching for rich screens
gRPC	Fast binary calls between services	High-performance internal microservices

For this guide: we concentrate on REST in Module 3, because it is the style you will bump into most often and the easiest one to reason about.

2.4 Why APIs Matter

It is worth pausing on why APIs earn so much attention. Three benefits stand out.

- **They decouple systems.** The client and the server can be built, released, and scaled on their own timelines. A team can quietly rewrite the backend in a new language, and as long as the API contract still holds, the clients never notice.
- **They enable reuse and integration.** One well-designed API can serve a website, a mobile app, internal tools, and outside partners all at once, so the logic is written once instead of being copied everywhere.
- **They power many kinds of app.** The same backend can drive a browser app, an iOS app, an Android app, and partner integrations at the same time, all through the same doorways.

2.5 Key API Concepts

A few terms come up constantly once you start working with APIs, so let us name them clearly.

- **Request and response payloads.** The payload is the body of data a request or response carries, for example the JSON describing a new user you are creating, or the JSON describing the user you get back.
- **Data formats, JSON and XML.** JSON, short for JavaScript Object Notation, is the format most modern APIs use: lightweight, easy to read, and natural in JavaScript. XML is older and more verbose, and it still turns up in enterprise and SOAP systems. Both describe structured data, but JSON is the default choice for most new web APIs.
- **API contracts and documentation.** The contract spells out every endpoint: its URL, its method, the inputs it expects, the outputs it returns, and the errors it can raise. Good documentation, often written with the OpenAPI or Swagger standard, is what lets other people use your API without reading your code.

The same data, written as JSON and as XML:

```
JSON:                          XML:
{                              <user>
  "id": 42,                      <id>42</id>
  "name": "Asha",                <name>Asha</name>
  "active": true                 <active>true</active>
}                              </user>
```

Module 3: RESTful APIs & Client-Server Architecture

3.1 Client-Server Architecture

REST sits on top of the client-server model, so it is worth making that model explicit before we go further.

- **Client and server.** The client starts the conversation by making a request, whether it is a browser, a mobile app, or another service. The server waits for requests and answers them. For any single interaction the roles are fixed: one side asks, the other replies.
- **How they talk.** They communicate over a network using an agreed protocol, which for web APIs is HTTP. The client sends a request message and the server returns a response message.
- **Statelessness.** Each request stands on its own and carries everything the server needs to handle it. The server does not remember earlier requests between calls, and that forgetfulness is actually a feature: it keeps the system simple and easy to scale.
- **Separation of concerns.** The client owns the interface and the experience, while the server owns the data, the rules, and the storage. Because the responsibilities are split cleanly, each side can change on its own.

3.2 What is REST?

REST stands for Representational State Transfer. It is an architectural style for designing networked applications, first described by Roy Fielding back in 2000. The important word there is “style”: REST is a set of guidelines, not a protocol and not a piece of software you install.

Two ideas sit at its heart. Everything is treated as a resource, such as a user, an order, or a product, and each resource has a URL that identifies it, for example /users/42. When you fetch a resource you do not get the resource itself; you get a representation of its current state, usually as JSON, which you can read or change and send back.

> **Key idea:** REST APIs are built around nouns. You act on resources (the nouns) using a small, fixed set of HTTP methods (the verbs), instead of inventing a brand new endpoint for every possible action.

3.3 REST Guiding Principles (Constraints)

An API is considered RESTful when it follows a handful of constraints. These are the ones worth knowing:

- **Client-server separation.** Interface concerns and data concerns are kept apart, so each side is free to evolve independently.
- **Statelessness.** Every request carries all the information needed to process it, and the server keeps no client session between requests. This is what makes REST scale so comfortably.
- **Cacheability.** Responses say whether they can be cached, so clients and the machines in between can reuse them and skip unnecessary work.
- **Uniform interface.** Resources are accessed in a consistent, predictable way, using resource URLs, standard HTTP methods, and standard status codes. This sameness is exactly what makes REST easy to pick up.
- **Layered system.** A client usually cannot tell whether it is talking straight to the server or through helpers like load balancers, caches, or gateways. Those layers can be added for scale and security without the client ever knowing.

3.4 HTTP Fundamentals for REST

REST rides on HTTP, so knowing HTTP well is most of the battle. Start with the methods, which act as the verbs of a request:

Method	Meaning	Typical use	Safe / Idempotent?
GET	Read a resource	Fetch a user or a list	Safe and idempotent
POST	Create a resource	Add a new order	Neither
PUT	Replace a resource	Full update of a record	Idempotent
PATCH	Partially update	Change one field	Not guaranteed

Method	Meaning	Typical use	Safe / Idempotent?
DELETE	Remove a resource	Delete an item	Idempotent

> **Two quick terms: idempotent** means calling something repeatedly has the same effect as calling it once, so deleting user 42 twice still just leaves the user deleted. **Safe** means the call does not change any data, which is true of GET. Both properties matter when you start dealing with retries and reliability.

Every response also comes with a status code, and its first digit tells you the headline at a glance:

- **2xx, success.** 200 OK, 201 Created, 204 No Content.
- **3xx, redirection.** 301 Moved Permanently, 304 Not Modified.
- **4xx, client error.** 400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found, 409 Conflict, 422 Unprocessable Entity.
- **5xx, server error.** 500 Internal Server Error, 502 Bad Gateway, 503 Service Unavailable.

Two more pieces round out an HTTP message. Headers carry metadata about the request or response, such as Content-Type: application/json, Authorization: Bearer <token>, Accept, or Cache-Control. The body carries the payload for methods like POST, PUT, and PATCH, while query parameters after a question mark refine a request, for example /products?category=books&page=2.

3.5 Designing RESTful Endpoints

Good endpoint design mostly comes down to a few habits.

- **Use resource-based URLs.** Prefer nouns, use plural names for collections, and keep them lowercase and nested: /users, /users/42, /users/42/orders. Avoid putting verbs in the path, so no /getUser.
- **Know path versus query parameters.** A path parameter points at one specific resource, as in /users/42. Query parameters filter, sort, or paginate a collection, as in /users?role=admin&sort=name.
- **Map CRUD onto methods.** The four basic data operations line up neatly with HTTP methods, shown in the table below.

Operation	HTTP method	Example endpoint
Create	POST	POST /users
Read	GET	GET /users and GET /users/42
Update	PUT or PATCH	PUT /users/42 or PATCH /users/42
Delete	DELETE	DELETE /users/42

- **Plan for versioning.** APIs change over time, so version them to avoid breaking existing clients. The common approaches are a prefix in the URL, such as /v1/users, or a version header. Introduce v2 alongside v1 rather than silently changing v1 underneath people.

3.6 Data Exchange in REST

JSON is the standard payload format for REST. A request carries JSON in its body, and the response returns JSON along with a status code. Here is a full round trip for creating a user.

The request:

```
POST /v1/users
Content-Type: application/json

{
  "name": "Asha Rao",
  "email": "asha@example.com",
  "role": "member"
}
```

A successful response:

```
201 Created
Content-Type: application/json

{
  "id": 42,
  "name": "Asha Rao",
  "email": "asha@example.com",
  "role": "member",
  "createdAt": "2026-07-30T09:15:00Z"
}
```

When something goes wrong, the error should be structured and helpful, not a raw stack trace dumped on the caller:

```
422 Unprocessable Entity
{
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Email is not valid.",
    "field": "email"
  }
}
```

> **Consistency pays off:** pick one shape for your successful responses and one for your errors, then use them everywhere. Clients can then handle every response the same way.

3.7 API Authentication & Security (Intro)

Security is a big topic, but a few basics will take you a long way.

- **API keys.** A simple secret string that identifies the calling app, sent in a header. Easy to use, but fairly blunt, so treat a key like a password.
- **Tokens.** After logging in, a client receives a token, often a JWT, and sends it on every request as Authorization: Bearer <token>. The server can verify it without storing a session, which keeps things stateless.
- **OAuth.** A standard framework that lets a user grant a third-party app limited access to their data without handing over their password. The “Sign in with Google” flow is the everyday example. It is more involved, but it is the norm for delegated access.
- **HTTPS, and why it matters.** Always serve APIs over HTTPS. It encrypts the request and the response as they travel, so credentials, tokens, and data cannot be read or altered by anyone on the network. Without it, a token can be plucked out of the air in plain sight.

> **A sensible baseline:** always use HTTPS, never put secrets in the URL, validate every input, and give each client only the access it genuinely needs.

Module 4: API Testing

4.1 Why Test APIs?

An API is a contract that a lot of clients lean on. If it breaks or quietly misbehaves, every app built on top of it breaks too. Testing is simply how we earn confidence that the contract still holds.

- **The payoff.** Tests catch bugs early, stop old bugs from creeping back in when code changes, document the behaviour everyone expects, and give the team the nerve to ship. Because an API has no screen of its own, testing it directly is often faster and more precise than poking at it through a user interface.
- **Where it fits.** API tests run throughout development: on your laptop while you code, automatically on every commit in the pipeline, and as health checks after a deploy. The earlier a problem is caught, the cheaper it is to fix.

4.2 Types of API Testing

Different kinds of testing answer different questions.

- **Functional testing.** Does each endpoint do the right thing for a given input? For example, does POST /users with valid data return 201 and the user that was just created?
- **Integration testing.** Do the endpoints work correctly together and with the real dependencies, like the database or other services? For example, does creating an order also reduce the stock count?

- **Performance and load testing.** How does the API hold up when many requests arrive at once? This measures response time and throughput and finds the point where things start to break.
- **Security testing.** Does the API turn away unauthorised access, resist common attacks such as injection, and avoid leaking sensitive data or internal stack traces?

4.3 What to Validate in an API

A good test checks more than just “did something come back.” Aim to confirm each of these:

- **Status codes.** The right code for the situation: 200 for a read, 201 for a create, 400 for bad input, 404 for something missing, and 401 or 403 when access is denied.
- **Response body.** The returned data has the fields, types, and values you expected.
- **Response headers.** The Content-Type is correct, along with any caching or custom headers that should be there.
- **Response time.** The call comes back within a threshold you are comfortable with.
- **Error handling and edge cases.** Missing fields, invalid values, empty inputs, and unauthorised calls all produce sensible, well-formed error responses rather than a crash.

4.4 Anatomy of an API Test

Almost every API test follows the same three beats: arrange the request, send it, and then check what came back.

5. Set up the request by choosing the method, the URL, the headers such as authentication, and the body.
6. Send the request and capture the response.
7. Assert on the response: check the status code, the body, the headers, and the timing against what you expected.

In practice it reads something like this:

```
// 1. Arrange
POST /v1/users
Header: Content-Type: application/json
Body:   { "name": "Asha", "email": "asha@example.com" }

// 2. Act: send the request, capture the response

// 3. Assert
expect status  == 201
expect body.id is a number
expect body.email == "asha@example.com"
expect responseTime < 500ms
```

4.5 Tools for API Testing (Overview)

You will reach for different tools depending on whether you are exploring or automating.

- **GUI tools such as Postman.** A visual app for building requests, grouping them into collections, inspecting responses, and adding checks. It is excellent for exploring and testing an API by hand. Insomnia is a similar alternative.
- **Command-line tools such as cURL.** A terminal command for firing off any HTTP request. It is perfect for quick checks, small scripts, and copy-paste examples in documentation.
- **Code-based frameworks.** Tests written in code that run automatically, for example REST Assured in Java, pytest with requests in Python, or supertest with Jest in Node.js. These are the ones you wire into your pipeline.

The same request in two tools:

```
# cURL
curl -X GET https://api.example.com/v1/users/42 \
     -H "Authorization: Bearer <token>"

# Postman: set method to GET, paste the URL, add the Authorization header,
#          click Send, then add a test: pm.response.to.have.status(200)
```

4.6 Introduction to Test Automation

- **Manual versus automated.** Manual testing, like clicking Send in Postman, is great when you are exploring. Automated testing runs the same checks over and over without any effort, so nothing quietly regresses as the code moves on.
- **Collections and suites.** Related tests are grouped together, for example all the tests about users, so they can be run and reported as one unit. Postman calls these collections, and code frameworks call them suites.
- **A word on CI.** Continuous Integration means your test suite runs automatically on every push or pull request. If a test fails, the change is flagged before it ever reaches users. This is the safety net that makes frequent changes feel safe.

> **The goal of automation:** turn “we think it works” into “the suite proves it works, on every change, in seconds.”

4.7 Best Practices

- **Cover the happy and the unhappy paths.** Test valid input that should succeed, and also invalid input, missing authentication, and not-found cases. Bugs love the edges.
- **Keep tests independent and repeatable.** Each test should set up its own data and not depend on the order of other tests or on leftover state. Running the whole suite twice should give the same result both times.
- **Write meaningful assertions and clear names.** Check what actually matters, such as the status, the key fields, and the error shape, and name each test so a failure reads like a sentence, for instance create_user_with_missing_email_returns_422.

Hands-On Practice

> **Goal:** make these ideas concrete by calling and testing a real API. Use any public API you like, such as a free weather API or JSONPlaceholder, and any tool you are comfortable with, whether that is Postman or cURL or check this Postman link.

1. In your own words, describe the journey of a request from a browser to a backend and back, naming DNS, IP addresses, HTTP or HTTPS, and ports along the way.
2. Using Postman or cURL, send a GET request to a public API and note the status code, the response body, and the Content-Type header.
3. On paper, design a small REST API for a notes app: list the resources, the endpoints, and the HTTP method for each create, read, update, and delete operation.
4. Write one positive and one negative test case for your create-note endpoint, stating the request and the expected response for each.
5. Bonus: trigger an error, for example by asking for a resource that does not exist, and describe the status code and error body you got back.

Gem

- Here is the gem link to ask questions and resolve doubts.

Reference Material

- MDN: An overview of HTTP
- MDN: HTTP response status codes
- REST API Tutorial
- Postman: What is API testing?

Advanced Reading Material

- Roy Fielding's dissertation, Chapter 5 on REST
- OpenAPI Specification
- OWASP API Security Top 10

Key Takeaways

- The backend turns requests into responses, applying business rules and touching data in between.
- An API is a contract: a stable, documented way for software to use a backend without knowing its internals.
- REST treats everything as a resource (a noun) acted on by a small set of HTTP methods (the verbs), and it leans on statelessness and a uniform interface.
- Use HTTP the way it was meant to be used: the right method, the right status code, JSON payloads, and consistent error shapes.

- Secure every API with HTTPS, proper authentication and authorization, and careful input validation.
- Test for the status, body, headers, timing, and error handling, then automate the suite and run it in your pipeline.